Introduction to Parallel Computing 2

Reference: “Introduction to Parallel Computing”, Chapters 1 and 2

Agenda

- Parallel algorithm design
  - Task decomposition
  - Mapping and scheduling
  - Communication and synchronization
- Performance modeling
  - Amdahl’s law
  - Gustafson's law

Parallel Algorithm Design

Creating a Parallel Program

◎ Thought process

1. Identify work that can be performed in parallel
2. Partition work (and also data associated with the work)
3. Manage data access, communication, and synchronization

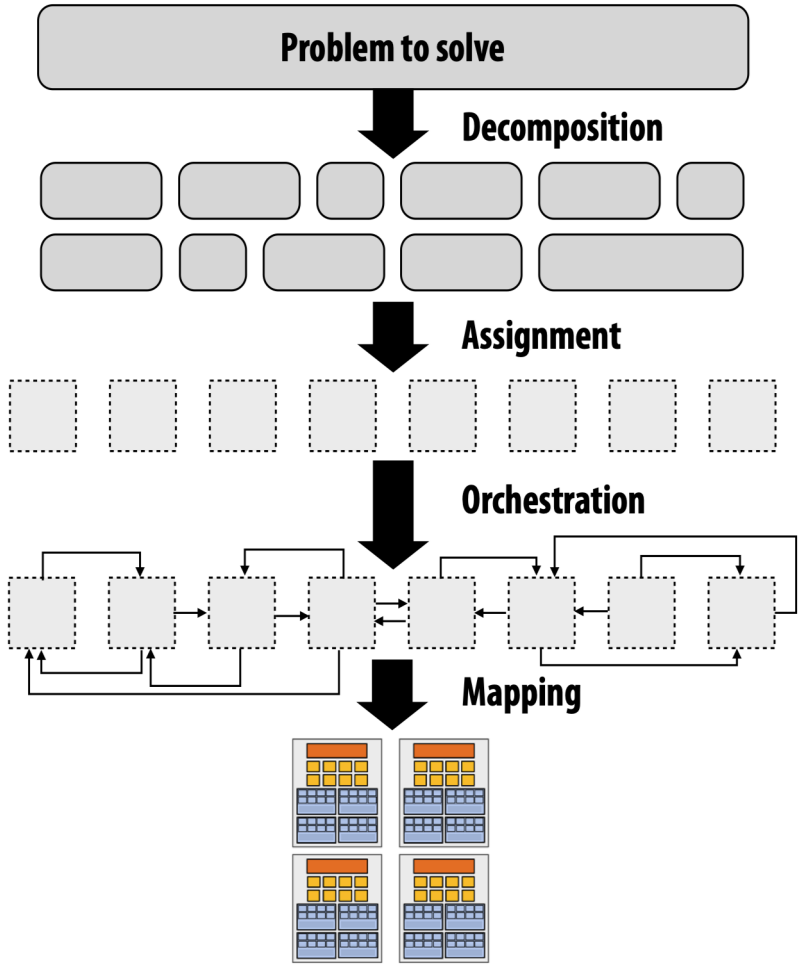
◎ One of our main goals is speedup

- For a fixed computation:

$$Speedup(P\ processors) = \frac{Time\ (1\ processor)}{Time\ (P\ processors)}$$

Steps in Creating a Parallel Program

"Mapping" is often automatically handled by OS, compiler, or HW



[Culler, Singh, Gupta 1998]

Elements of a Parallel Algorithm

◎ Task decomposition

- Partition the problem into pieces of work that can be executed concurrently

◎ Mapping and scheduling

- Map the tasks onto multiple computing units
- Distribute input/output and intermediate data

◎ Communication and synchronization

- Synchronize the tasks at various points of the parallel execution

Holy Grail of Parallel Algorithm Design

◎ Maximize parallelism, i.e., number of tasks that can be executed in parallel at any point of time

◎ Minimize communication

◎ Minimize load imbalance

→ Maximize potential speedup!

◎ Find a sweet-spot that balances out conflicting goals

- Parallelism vs. communication
- Load imbalance vs. communication
- Considering architectural constraints

Holy Grail of Parallel Algorithm Design

◎ Maximize parallelism, i.e., number of tasks that can be executed in parallel at any point of time

◎ Minimize communication

◎ Minimize load imbalance

"Art of parallel programming"
How to apply different rules and skills for different algorithms and architectures

◎ Find a sweet spot that balances out conflicting goals

- Parallelism vs. communication
- Load imbalance vs. communication
- Considering architectural constraints

Task Decomposition

- **Divide the problem into discrete “chunks” of work, i.e., tasks, to be carried out in parallel**
  - Usually many ways to decompose exist
  - Task sizes may not be uniform
- Main idea: create at least enough tasks to keep all execution units on a machine busy

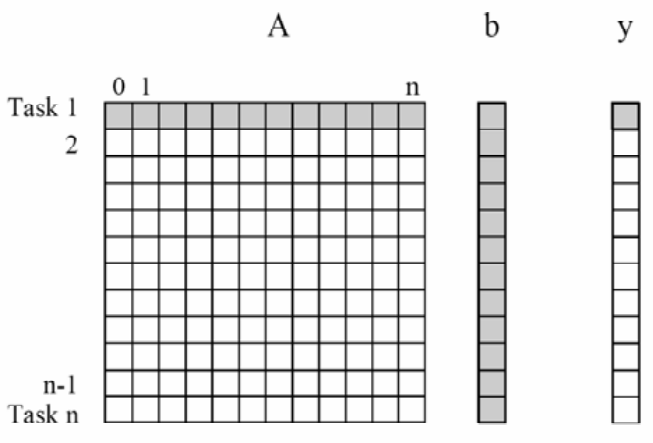
Key aspect of decomposition:

identifying dependencies (or a lack thereof)

Example: Task Decomposition

```
int A[n][n];
int x[n], y[n];
...
void mvm(int **A, int *x, int *y) {
    for (i=0; i<n; i++) {
        y[i] = 0
        for (j=0; j<n; j++) {
            y[i] += A[i][j] * x[j]
        }
    }
}
```

Dense matrix-vector multiplication



Decomposed into n tasks
where n is the number of rows in the matrix

Example: Database Query Processing

MODEL = “CIVIC” AND YEAR = 2001 AND
(COLOR = “GREEN” OR COLOR = “WHITE)

Consider the execution of the query

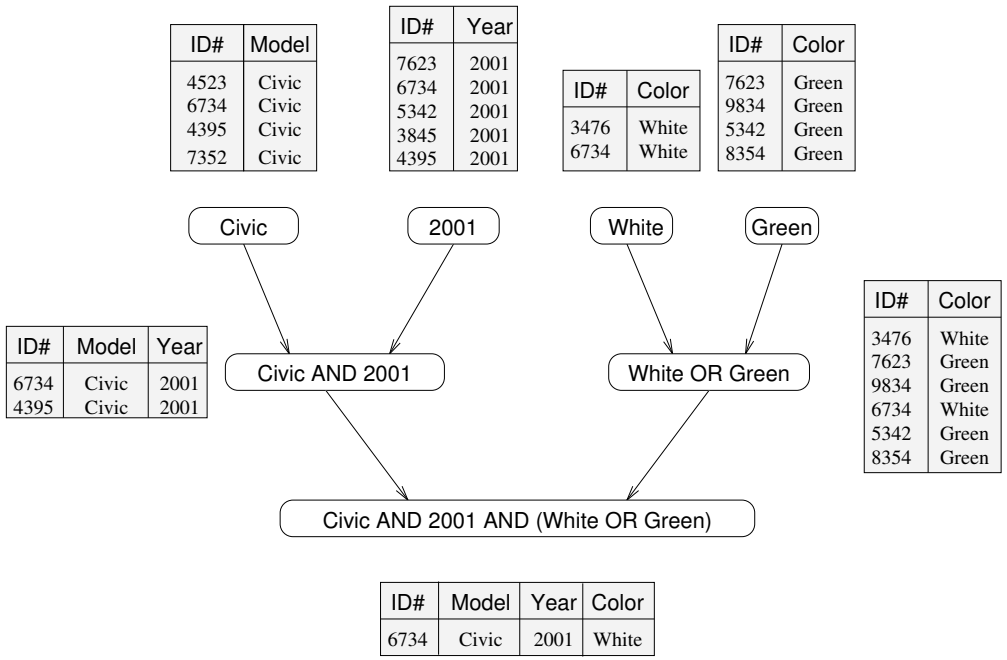
ID#	Model	Year	Color	Dealer	Price
4523	Civic	2002	Blue	MN	$18,000
3476	Corolla	1999	White	IL	$15,000
7623	Camry	2001	Green	NY	$21,000
9834	Prius	2001	Green	CA	$18,000
6734	Civic	2001	White	OR	$17,000
5342	Altima	2001	Green	FL	$19,000
3845	Maxima	2001	Blue	NY	$22,000
8354	Accord	2000	Green	VT	$18,000
4395	Civic	2001	Red	CA	$17,000
7352	Civic	2002	Red	WA	$18,000

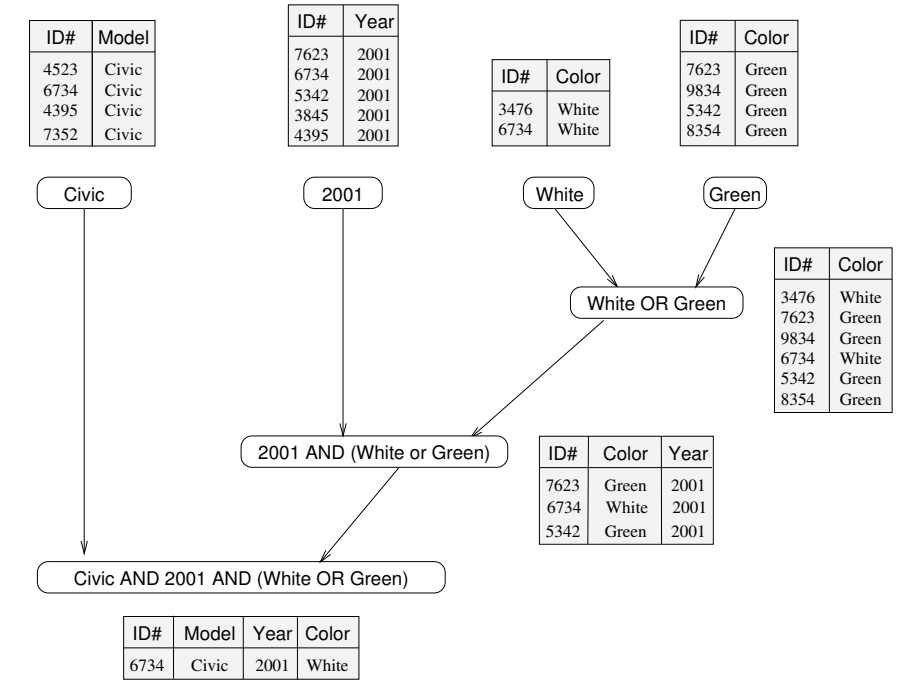
On the database

Example: Task Decomposition

- Data decomposition
  - Identify the data on which computations are performed
  - Partition this data across various tasks.
  - This partitioning induces a decomposition of the problem
  - Data can be partitioned in various ways – this critically impacts performance of a parallel algorithm
- Output data decomposition
  - Each element of the output can be computed independently of others
- Input data decomposition
  - Generally applicable if each output can be naturally computed as a function of the input
  - In many cases, this is the only natural decomposition because the output is not clearly known a-priori
    - Finding the minimum in a list, sorting a given list, etc.

Example: Database Query Processing





- Query execution can be divided into subtasks in various ways
  - Each task generates an intermediate table of entries
  - Edges in the graph denote the output of one task is needed to accomplish the next

→ Different task decompositions may lead to significant performance difference

Critical Path Length

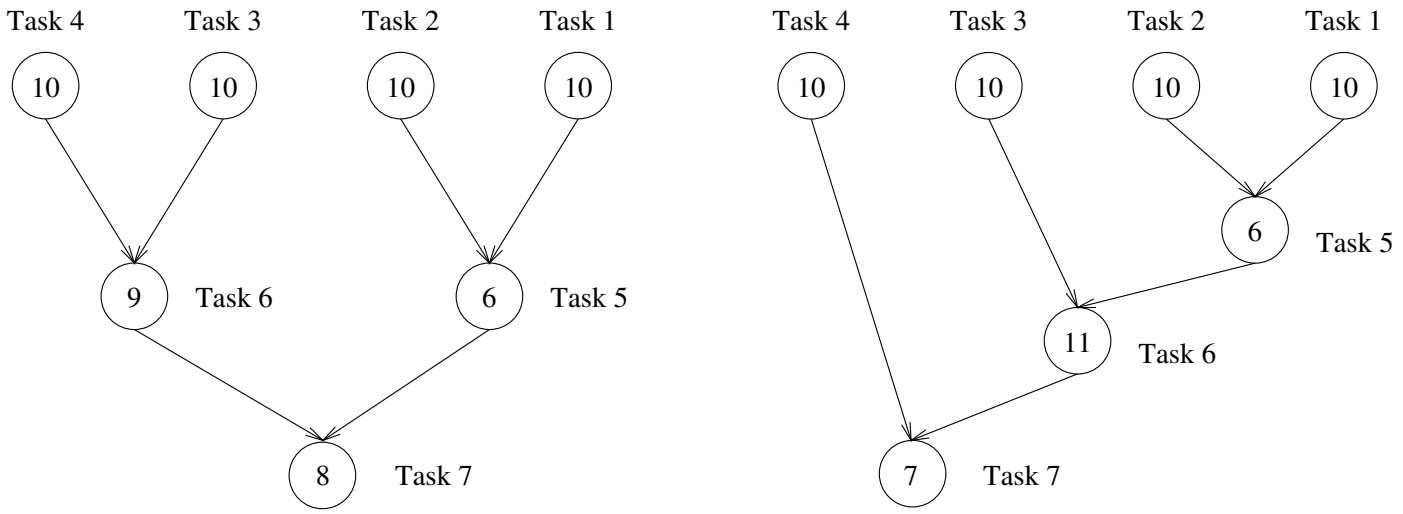
- A directed path in the task dependency graph represents a sequence of tasks that must be processed one after the other

- The longest such path determines the shortest time in which the program can be executed in parallel

- The length of the longest path in a task dependency graph is called the critical path length

Critical Path Length

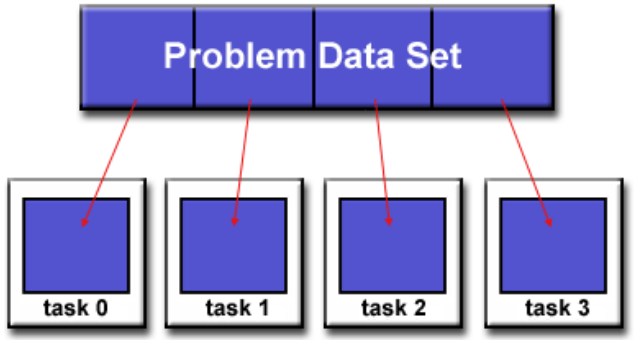
- What are the critical path lengths for the two task dependency graphs?
- If each task takes 10 time units, what is the shortest parallel execution time?
- How many processors are needed in each case to achieve the minimum parallel execution time?



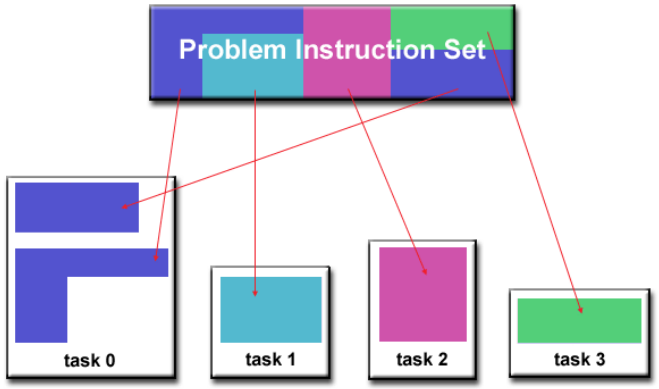
Task Decomposition

◎ How to decompose?

- Domain decomposition
  - Data associated with a problem is decomposed
  - Each parallel task then works on a portion of the data
  - E.g., data-parallel algorithms



- Functional decomposition
  - Work that must be done for the problem is decomposed
  - Each task then performs a portion of the overall work
  - E.g., pipeline parallel algorithms



Task Decomposition

◎ When to decompose?

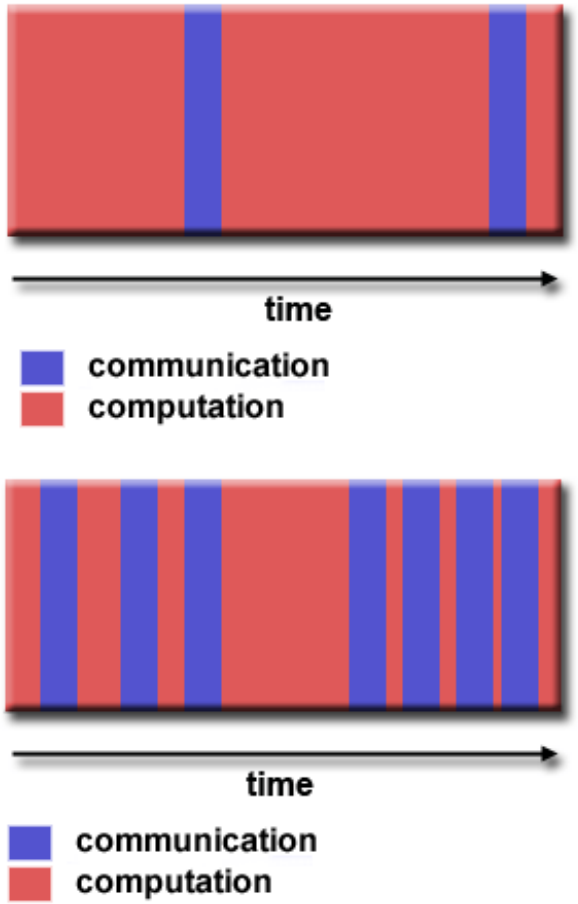
- Static decomposition
  - Decide the decomposition at the beginning of the program computation
  - Applicable when tasks are known

- Dynamic decomposition
  - Decide the decomposition dynamically, based on the input characteristics
  - Can increase overheads
  - E.g., sparse-matrix multiplication

Task Decomposition

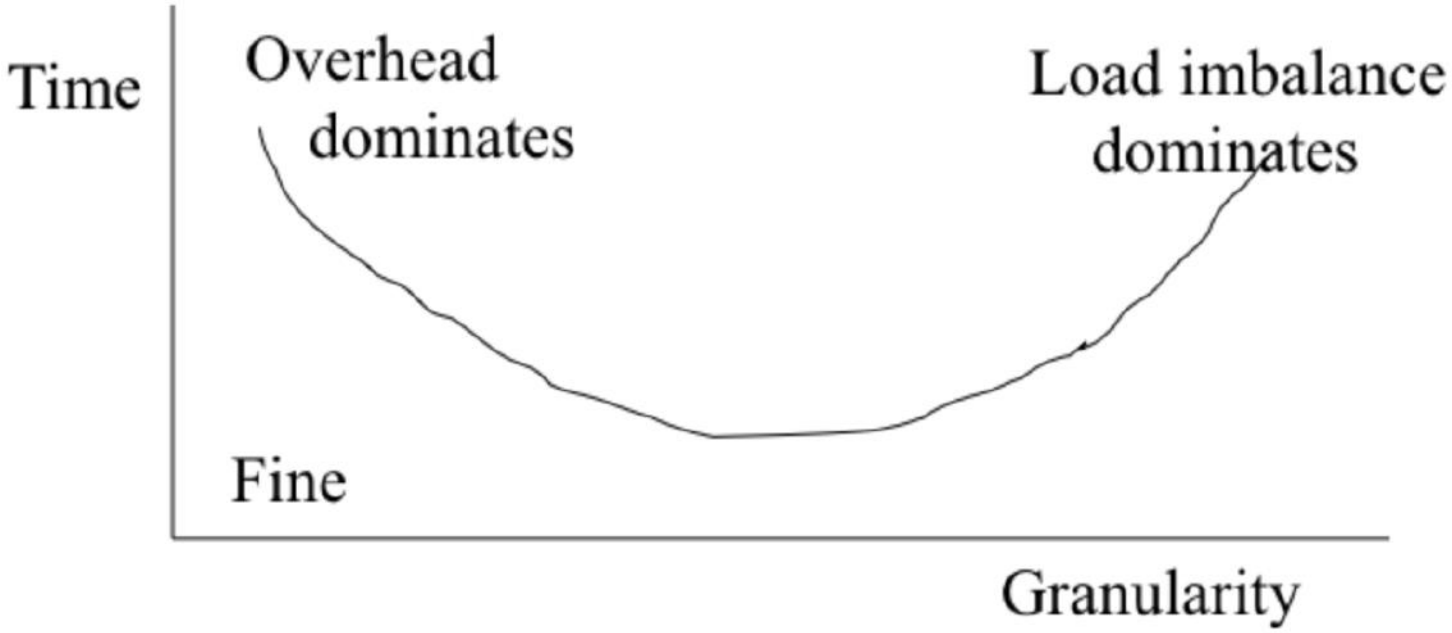
◎ How large/small are tasks?

- Related to task size and “computation and communication ratio”
- Coarse-grained tasks
  - Small # of tasks
  - High computation and communication ratio; relatively easier to improve performance
  - Harder to load balance efficiently
- Fine-grained tasks
  - Large # of tasks
  - Facilitate load balancing
  - High communication overheads



Task Decomposition

- The most efficient granularity depends on the algorithm and the hardware
  - Parallel execution overheads = communication/synchronization + idling + excess work



Mapping and Scheduling

◎ Determine the assignment of the tasks to processing units (**mapping**) and the timing of their execution (**scheduling**)

◎ A proper mapping is critical as it minimizes the parallel execution overheads

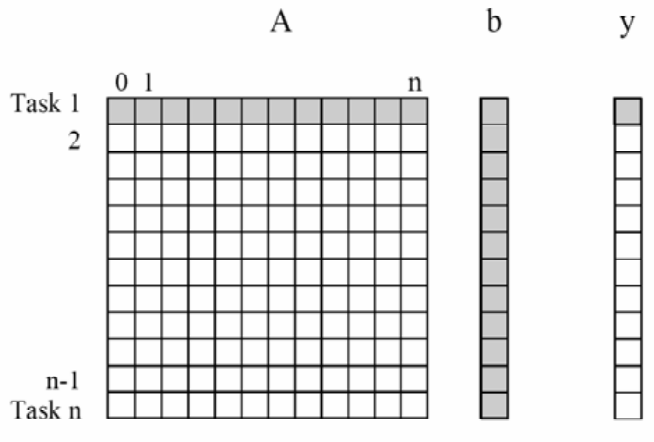
- The total overhead $T_O = p * T_p - T_s$ where T_p = parallel runtime on p processors and T_s is the serial runtime
  - The work done by the parallel system beyond that required by the serial system
- Overhead sources
  - Load imbalance
  - Communication/synchronization
  - Excess work

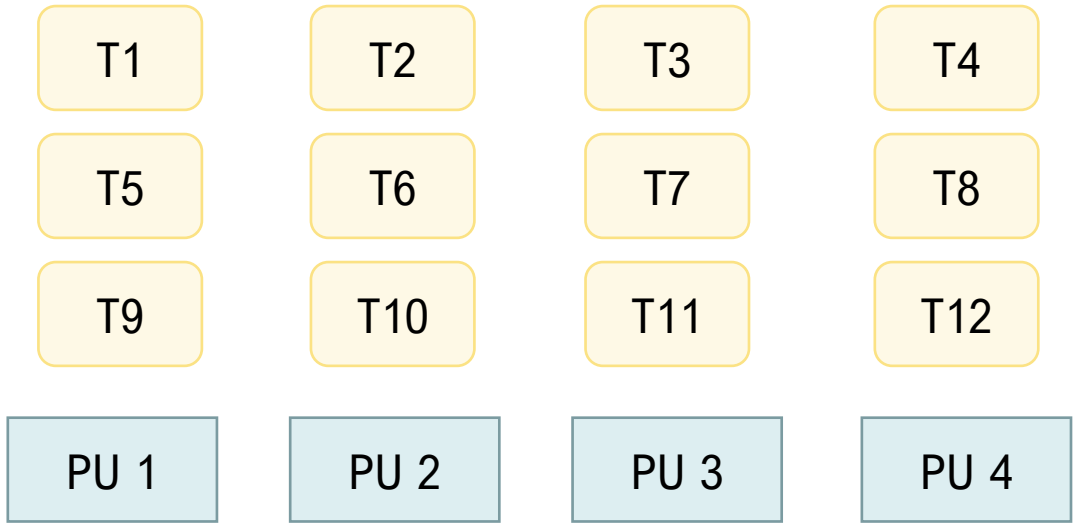
Mapping and Scheduling

- Static mapping and scheduling
  - Tasks are statically assigned to processing units before the execution
  - Applicable if tasks are generated statically and task size is constant
  - Minimize overhead

- Dynamic mapping and scheduling
  - Tasks are dynamically assigned to processing units during execution
  - E.g., tasks queue, task/data migration, ...
  - Used when tasks are generated dynamically and have unknown computational requirements
  - May incur higher overheads, but may reduce load imbalance

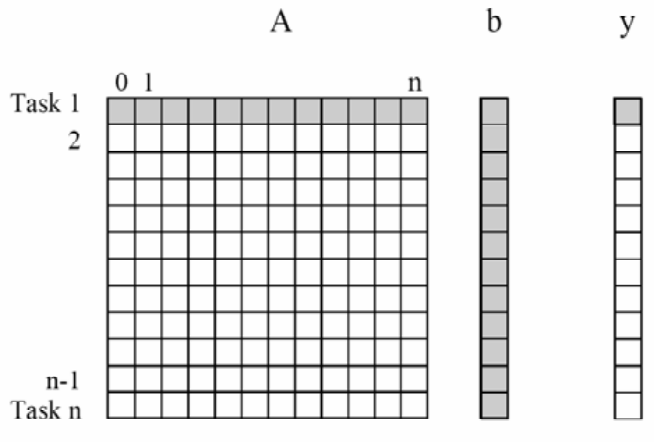
Mapping and Scheduling

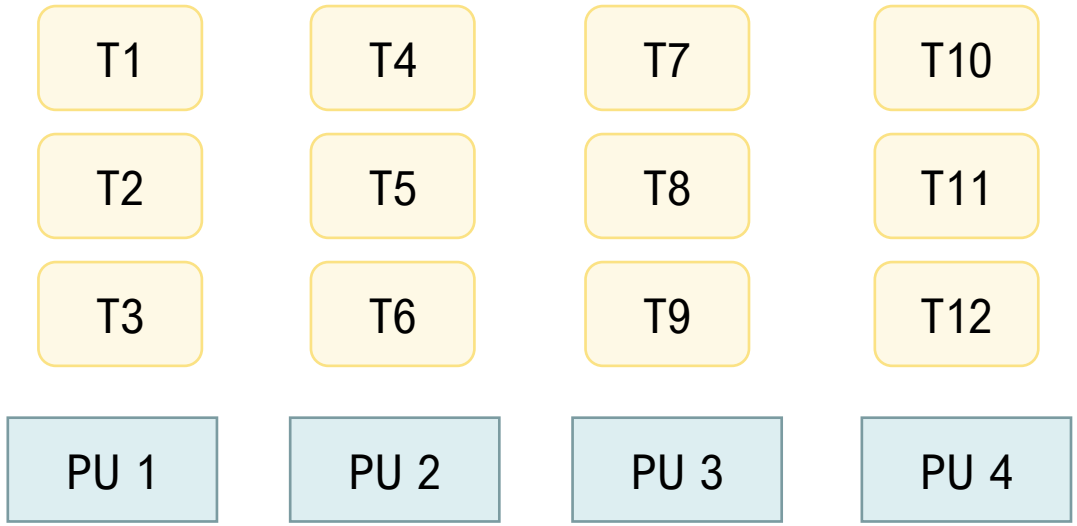




Static decomposition
Static mapping
Locality?

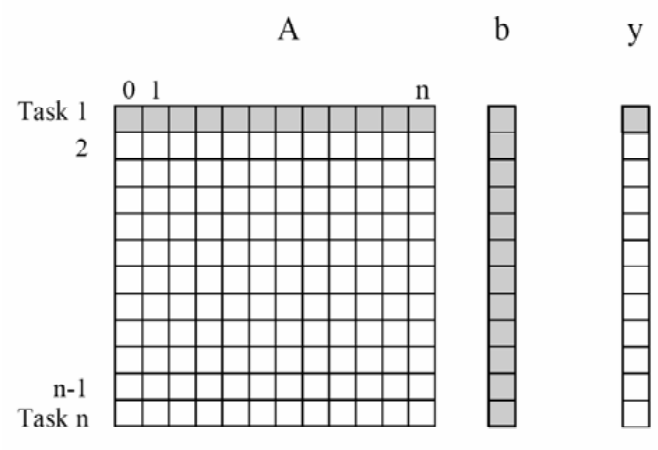
Mapping and Scheduling





Static decomposition
Static/dynamic mapping (affinity)
Better locality

Mapping and Scheduling

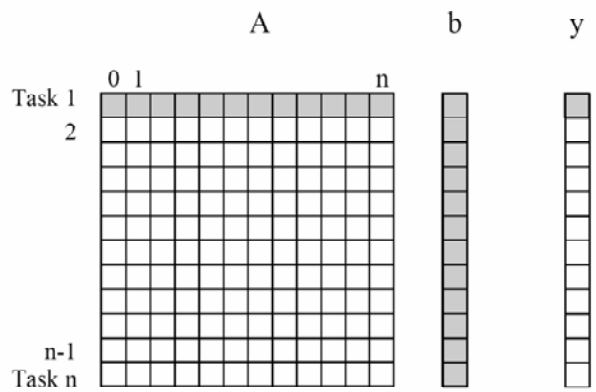


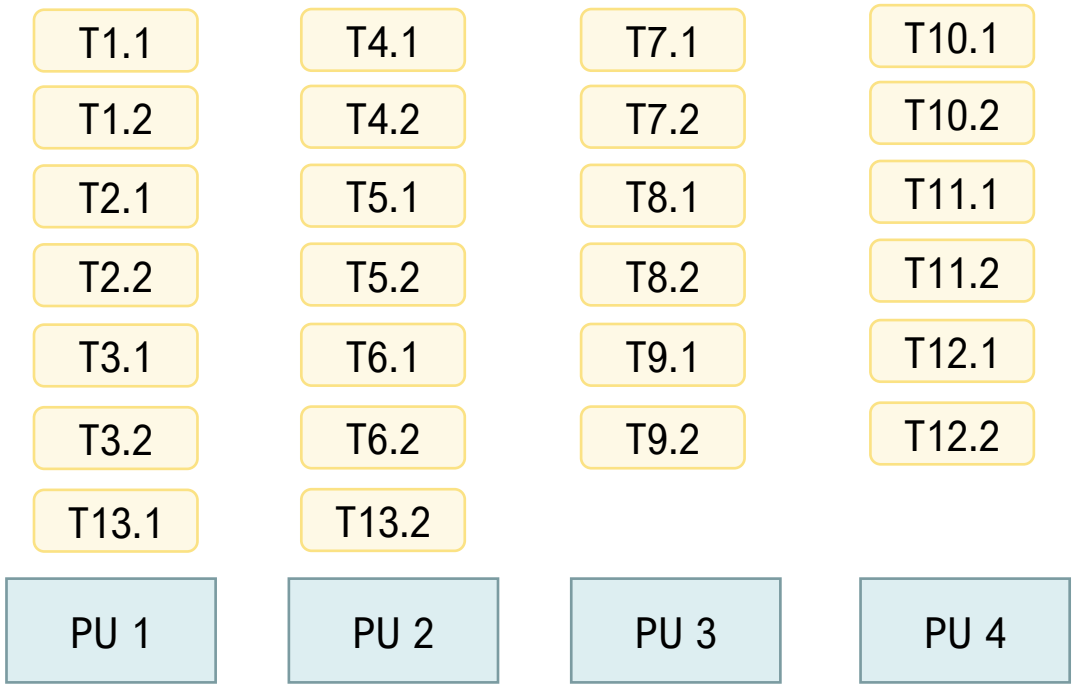
Could be a critical path
Determines parallel execution time

T1	T4	T7	T10
T2	T5	T8	T11
T3	T6	T9	T12
T13			
PU 1	PU 2	PU 3	PU 3

Static decomposition
Static/dynamic mapping
Load imbalance

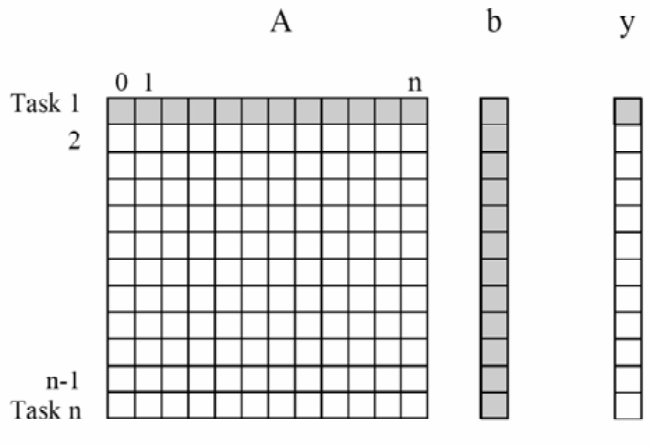
Mapping and Scheduling





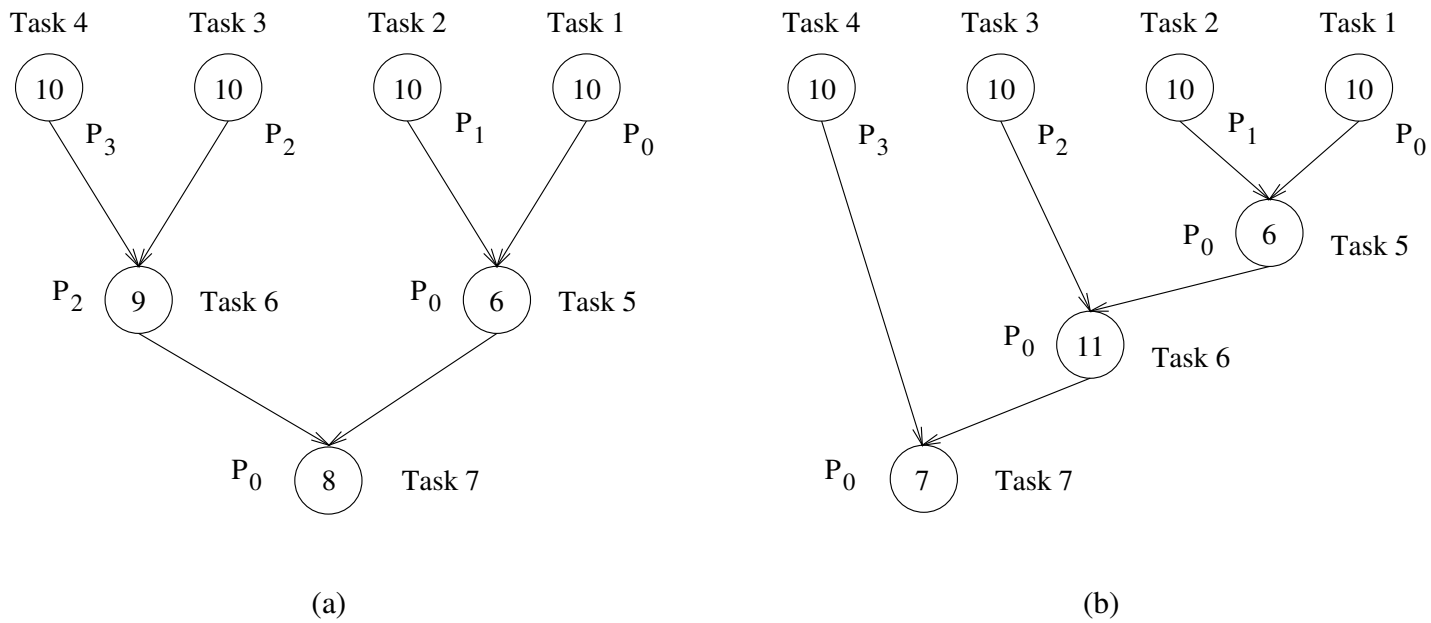
Static decomposition
Static/dynamic mapping
Finer-grained tasks

Mapping and Scheduling



Static/dynamic decomposition
Dynamic mapping

Mapping and Scheduling



Tasks within a single level are assigned to different tasks

Mapping and Scheduling

◎ Ways to minimize overheads

- Map independent tasks to different processing units
  - Maximize parallelism
- Assign tasks on critical path to processing units as soon as they become available
  - Minimize load imbalance
- Minimize communication between processing units by mapping tasks with dense interactions to the same processing units
  - Minimize overheads

These criteria often conflict with each other

Communication and Synchronization

- Sharing data between parallel tasks
- **Communication is an overhead that does not exist in a sequential program**
  - Cycles spent to prepare, send and receive shared data
  - Communication requires some form of synchronization between tasks, which can result in tasks waiting instead of doing useful work
  - Contention from competing communication can saturate the network bandwidth and increase synchronization overhead
- The need for communication between tasks depends upon the problem
  - Little or not communications are required → embarrassingly parallel
  - Most parallel algorithms require tasks to share data

Communication and Synchronization

◎ **Data sharing method**

- Message passing model
  - Communications are explicit and under the control of the programmer
  - Higher programming burden, less flexible, but easier to understand

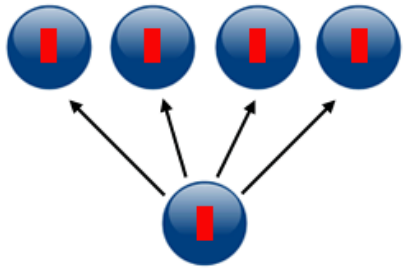
- Shared memory model
  - Communications are implicit and transparent to the programmer
  - Need software/hardware support for communication under the hood (e.g., coherence protocol)
  - Easier to program, but more prone to bugs

◎ **Programming models and hardware implementation can differ**

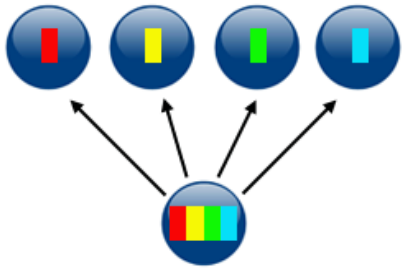
Communication and Synchronization

◎ Types of communication

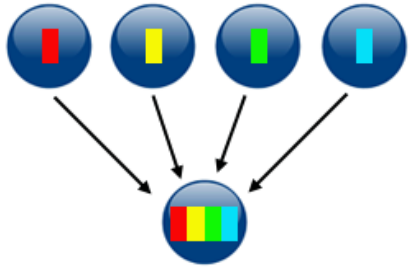
- Collective
  - Broadcast (one-to-all, all-to-all)
  - Reduction (all-to-one)
  - Scatter and gather
- Point-to-point
  - Producer-consumer



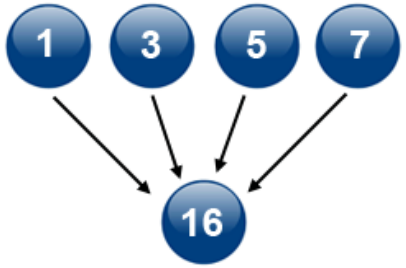
broadcast



scatter



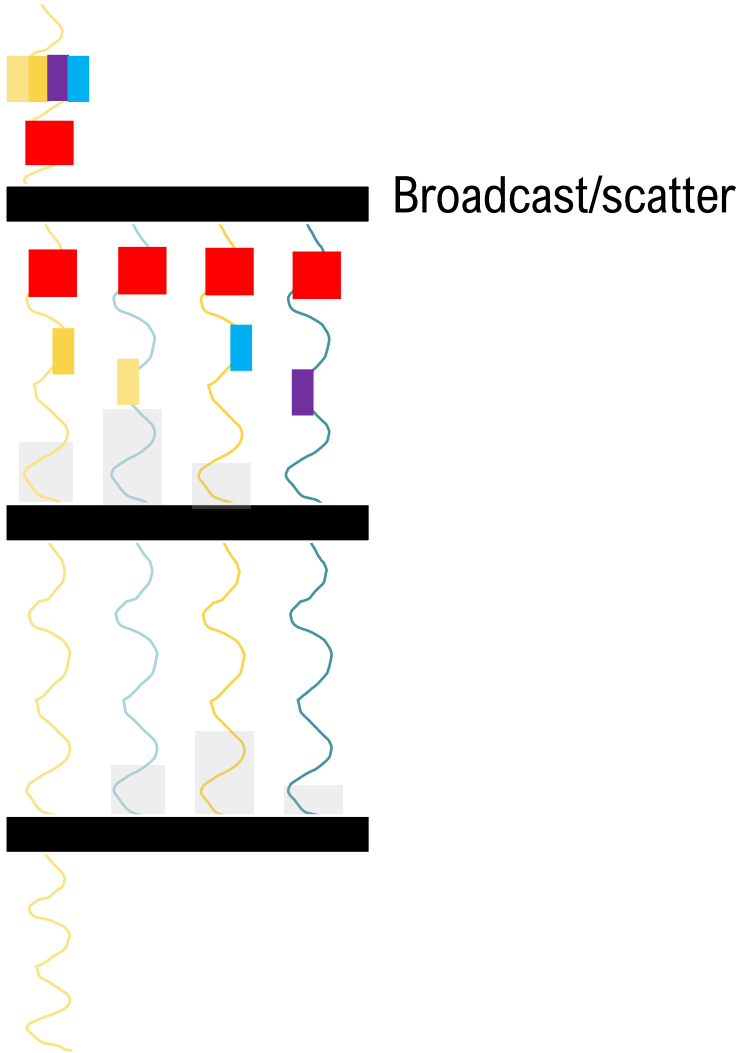
gather



reduction

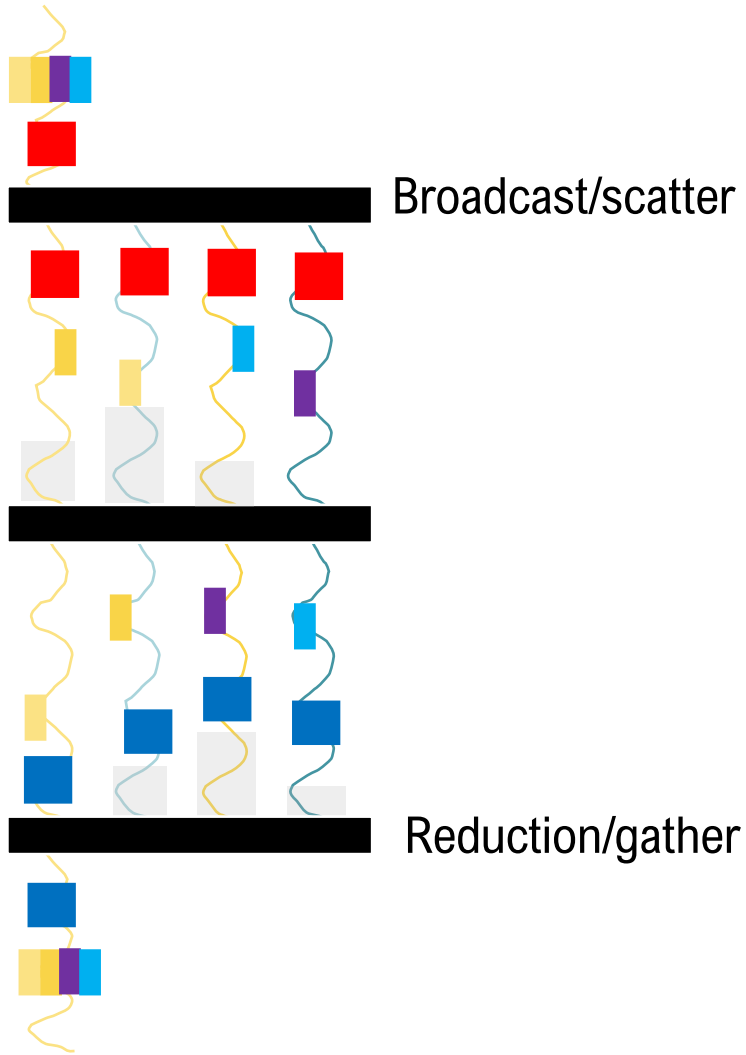
Communication and Synchronization

- Collective



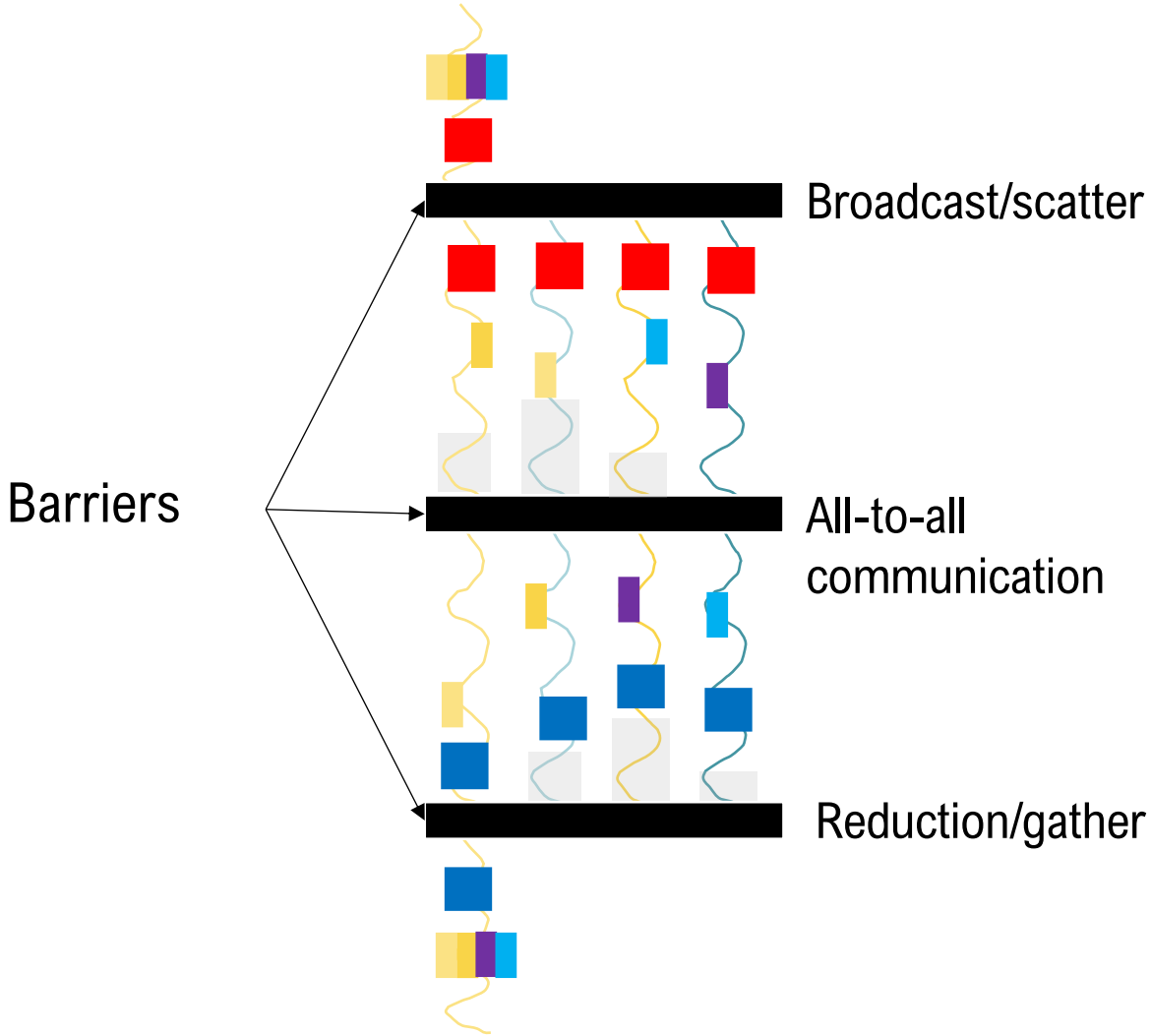
Communication and Synchronization

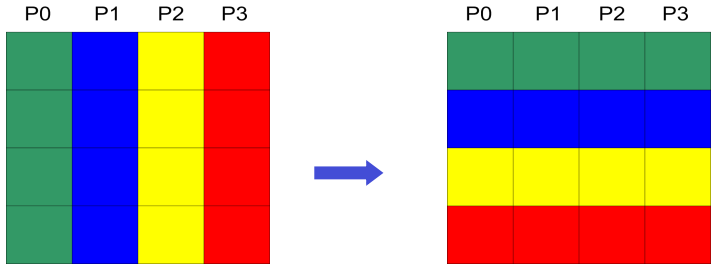
- Collective



Communication and Synchronization

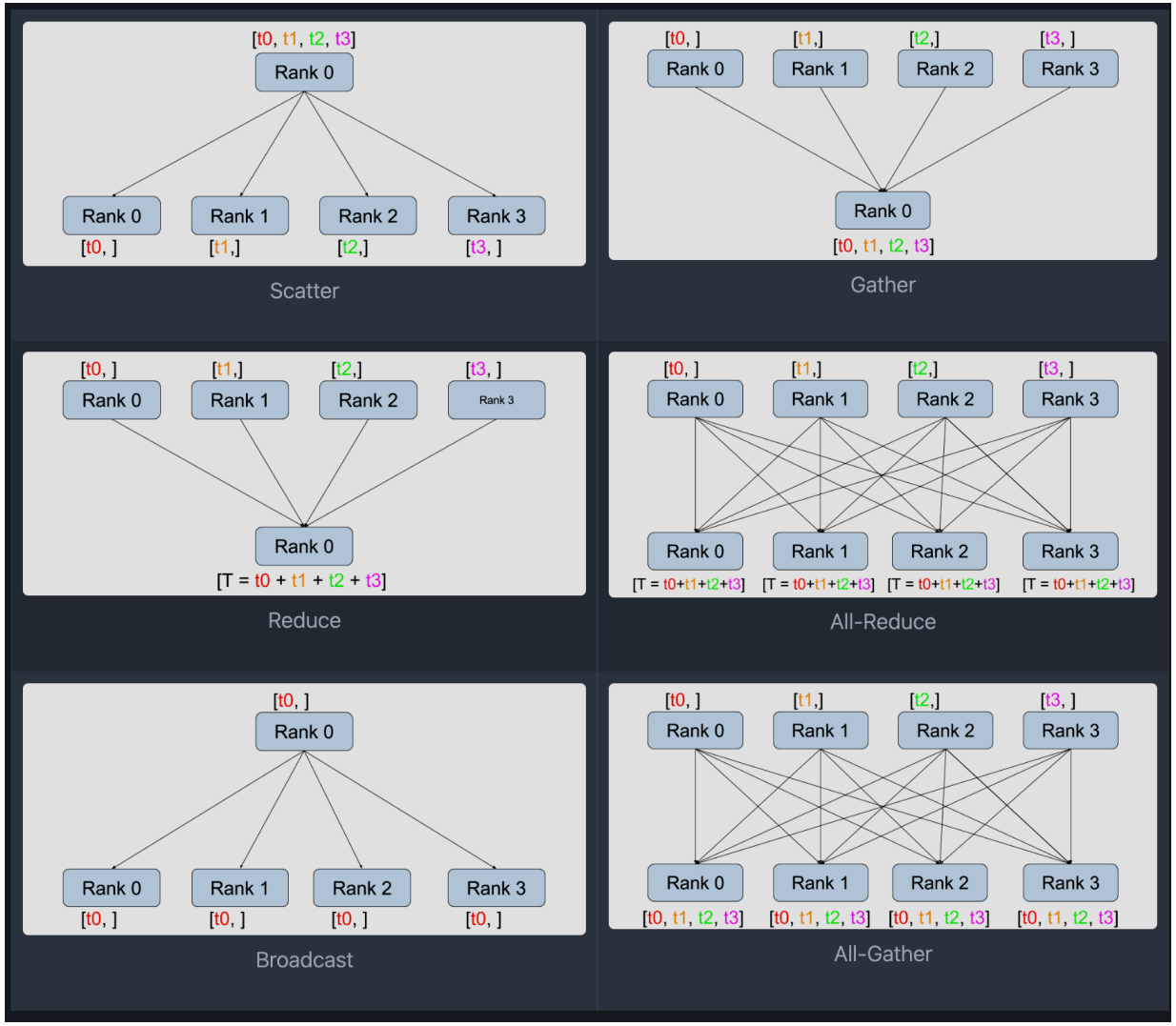
- Collective





Collective Communication

- All-reduce
  - Obtain th sum of all data (tensors) on all processes
- All gather
  - Collect all data (tensors) on all processes
- E.g., Distributed model training
  - Distribute mini-batches, model layers, GEMM computations along rows or columns
  - Use synchronization among workers to gather results



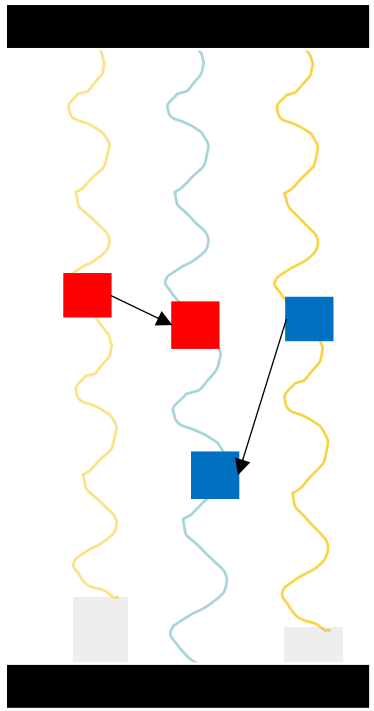
https://docs.pytorch.org/tutorials/intermediate/dist_tuto.html#setup

Barrier Synchronization Primitives

- Barriers are a conservative way to express dependencies
- Barriers divide computation into phases
- All computation by all threads before the barrier complete before any computation in any thread after the barrier begins
  - All computations after the barrier are assumed to depend on all computations before the barrier

Communication and Synchronization

- Point-to-point
  - Specify fine-granularity dependencies
  - Reveal more parallelism than barriers



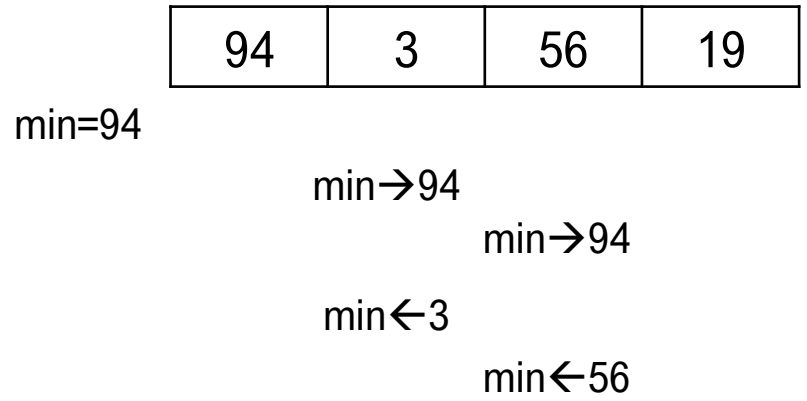
Locks
Atomic updates
Transactions
...

Serialization of Shared Data Accesses

```
int A[n];
int min;
...
int serial_min(int *A, int n) {
    min = A[0];
    for (i=0; i<n; i++) {
        if (A[i] < min) min = A[i]
    }
    return min
}
```

Minimum finding

min is shared between parallel tasks
Read and write accesses to the value of min should be serialized
Otherwise, the result can be incorrect

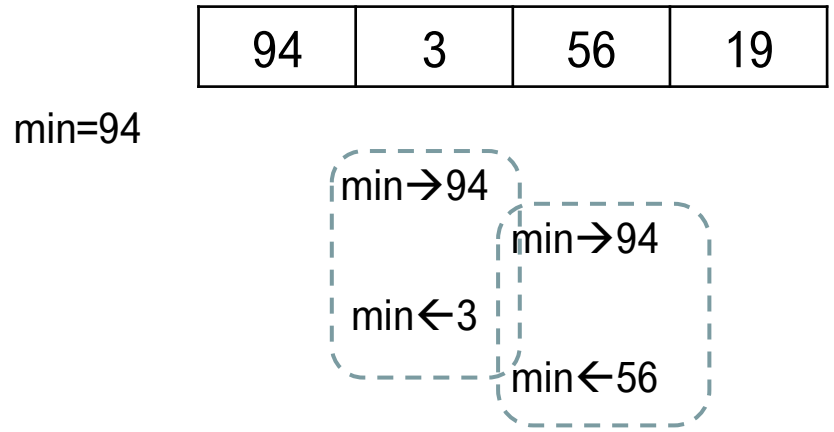


Serialization of Shared Data Accesses

```
int A[n];
int min;
...
int serial_min(int *A, int n) {
    min = A[0];
    for (i=0; i<n; i++) {
        if (A[i] < min) min = A[i]
    }
    return min
}
```

Minimum finding

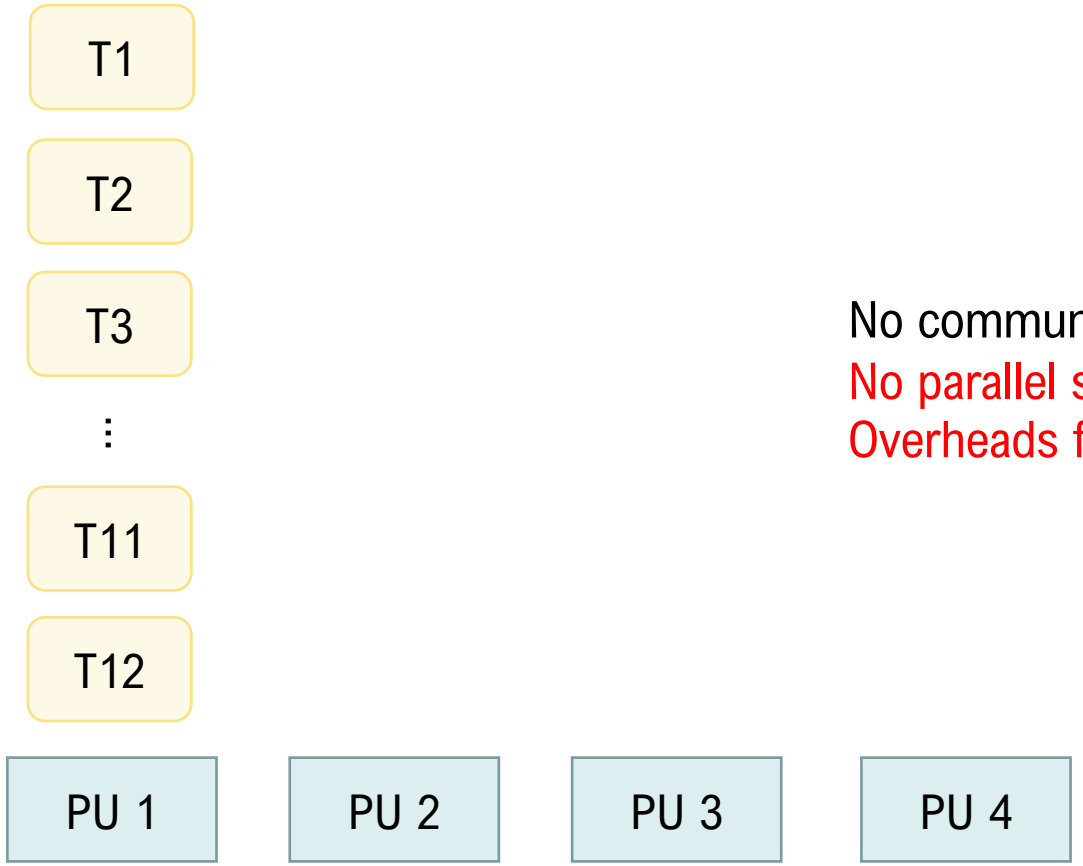
min is shared between parallel tasks
Read and write accesses to the value of min should be serialized
Otherwise, the result can be incorrect



Critical section
Atomically executed

Communication Overhead



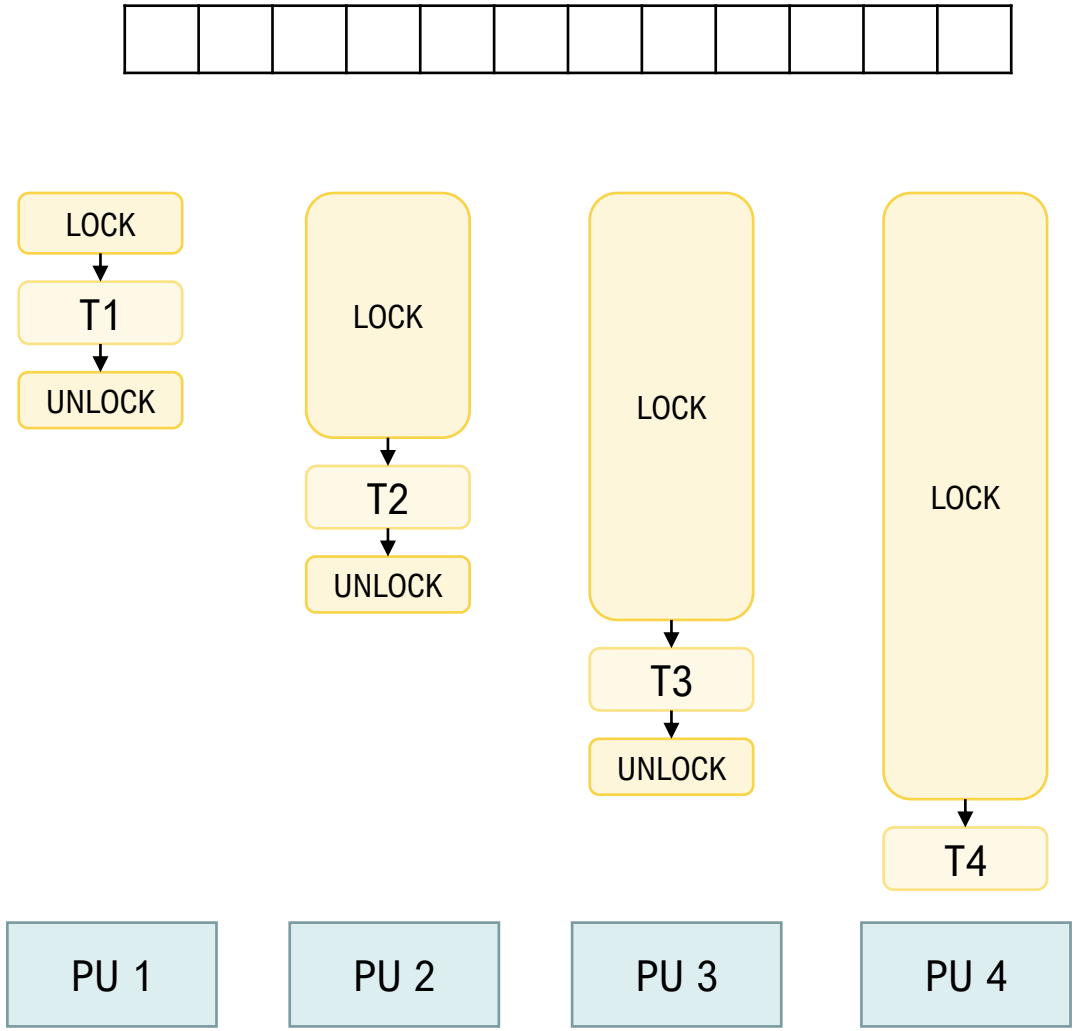


No communication overheads

No parallel speedup

Overheads from parallel task creation

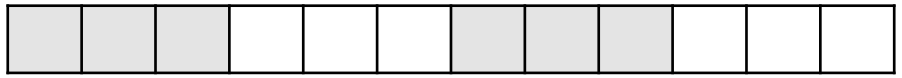
Communication Overhead

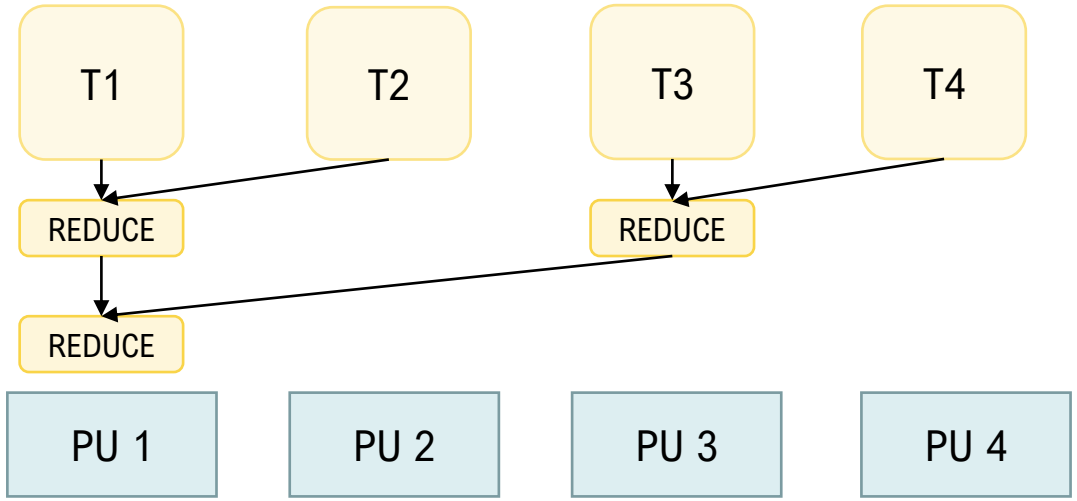


Task granularity = one iteration
Centralized communication

Communication overheads
= synchronization time
(including atomic accesses and waiting time)
+ data communication time
(for shared variable)

Communication Overhead





Task granularity = n/4 iteration
Map/reduce communication
(distributed reduction)

Communication overheads
= reduction time
(distributed synchronization)
+ data communication time
(for intermediate reduction results)
Extra memory for local reduction results

Communication and Synchronization

◎ **In modern machines, communication is much more expensive than computation**

- Both in terms of performance and power

◎ **Ways to minimize communication overheads**

- Maximize data locality
  - Restructure computation so that data can be reused
- Minimize volume of data exchange
  - There is a cost associated with each word that is communicated → minimize the volume of data communicated
- Minimize frequency of communications
  - There is a startup cost associated with each communication → merge multiple interactions into one
- Minimize contention and synchronization hot-spots
  - Use decentralized techniques and replicate data when necessary
- Overlap computation and communication
  - Non-blocking communication

Performance Modeling

Basic Measures of Parallelism

- **Parallel execution time**
  - T_p: time spent to solve a problem on p processors
- **Total overhead function**
  - $T_o = pT_p - T_s$
- **Speedup**
  - $S_p = {}^{T_s}/_{T_p}$: sequential execution time / parallel execution time
- **Efficiency**
  - $E = {}^{S_p}/_{p} = {}^{T_s}/_{p \times T_p}$: measures how much extra work the parallel execution does
  - If $E > 1$, the speedup is superlinear

Amdahl's Law

◎ If a change improves a fraction f of the workload by a factor of K (=p)

$T = (1 - f)T + fT$ The execution time before an improvement

$T_K = (1 - f)T + \frac{f}{K}T$ The execution time after an improvement

◎ f is the fraction that can be run in parallel

◎ Fraction $(1 - f)$ should be run sequentially

Amdahl's Law

◎ Speedup in latency

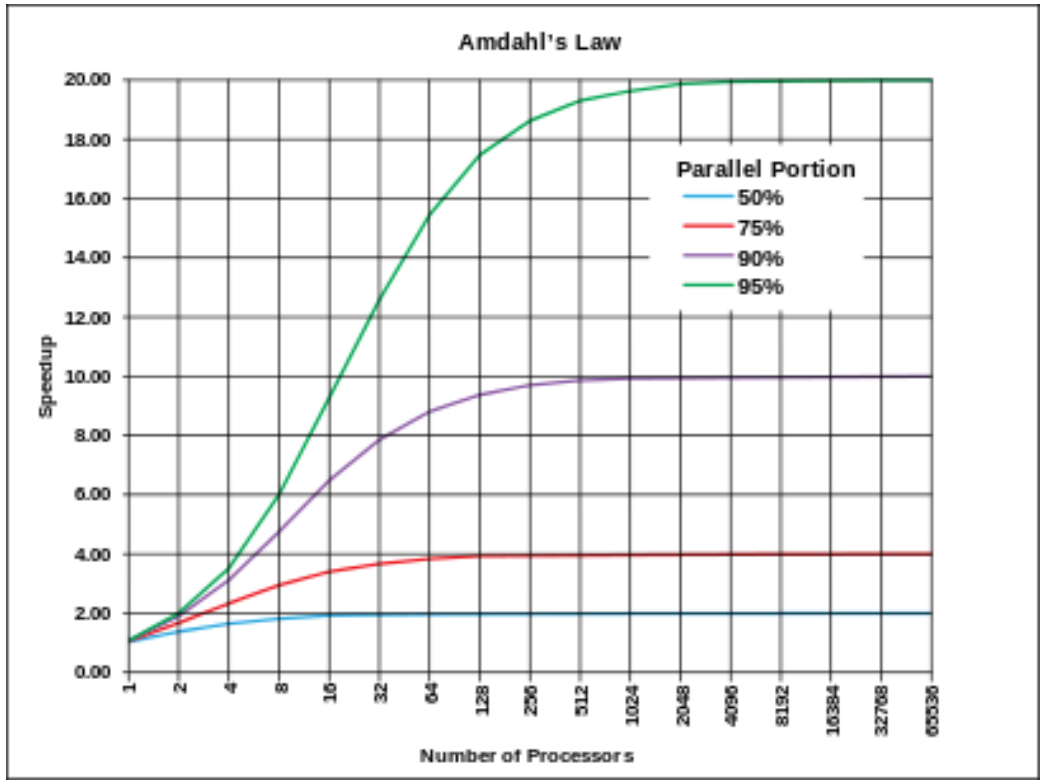
$$S = \frac{T}{T_K} = \frac{1}{(1-f) + \frac{f}{K}}$$

1. Look for algorithms with large *f*

- Otherwise, do not bother with parallelism for performance
- Speed up is limited by sequential code
- Even a small percentage of sequential code can greatly limit potential speedup

Amdahl's Law

Speed up for different values of f



$$S_\infty = \frac{1}{(1 - 0.95)} = 20$$

$$S_\infty = \frac{1}{(1 - 0.5)} = 2$$

Amdahl's Law

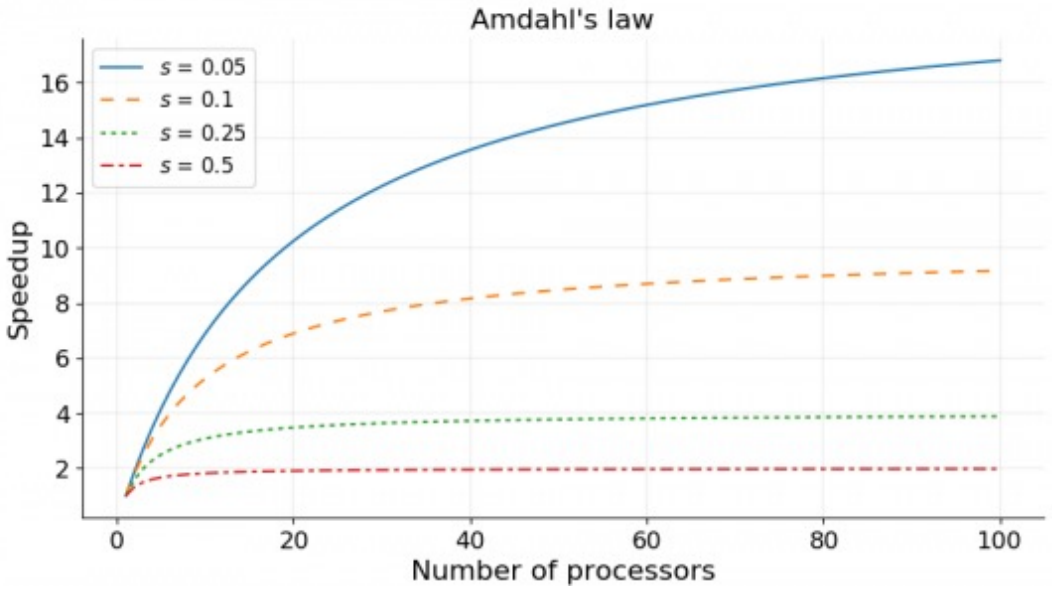
$$S = \frac{1}{(1-f) + \frac{f}{K}}$$

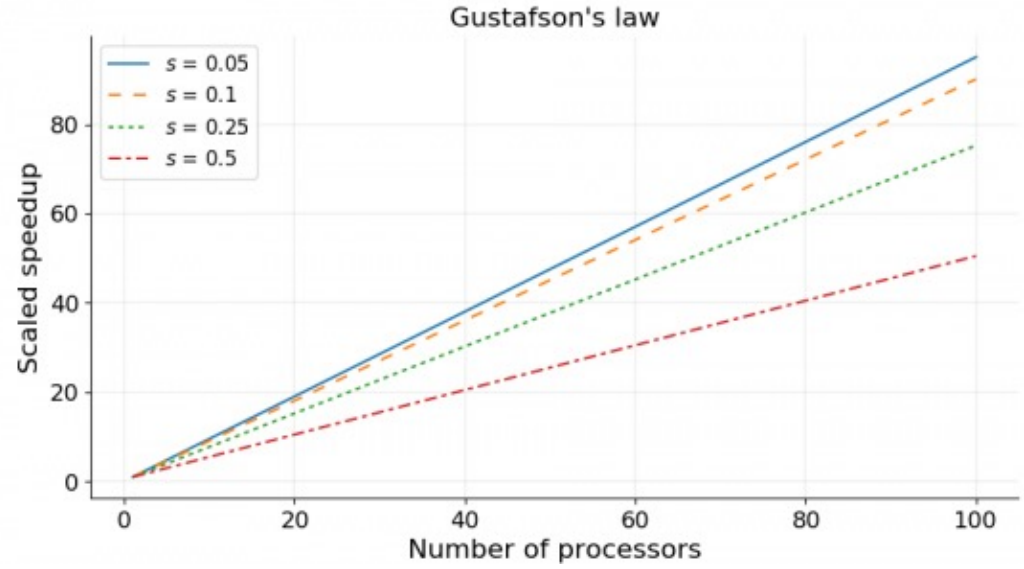
2. Look for algorithms with large K

- Data-parallel computation
  - Perform the same operation to different items of data at the same time
  - With a large set of data, K scales well with the size of the data
- Task-parallel computation
  - Perform distinct operations (tasks) at the same time
  - With the # of tasks fixed, K does not scale very well
- A combination of the two is increasingly common

Strong vs. Weak Scaling

- **Strong scaling**: How does E_p vary with P when the problem size is fixed?
  - E.g., how much speedup with a 100x100 matrix multiplication with increasing # of processors?
  - Fixed-sized computations are only scalable up to a certain processor count
- **Weak scaling**: How does E_p vary with P when the problem size per processor is fixed
  - i.e., the problem size grows linearly with P





Gustafson's Law

◎ **If you can increase the amount of work done by each process/task, then the serial component will <u>not</u> dominate**

- Increase the problem size to maintain scaling in terms of adding extra complexity or increasing the overall problem size
- The parallel part scales with the resources, while the serial part does not increase
- $T = (1 - f)T + fKT$
- $T_K = (1 - f)T + \frac{f}{K}KT$
- $S = \frac{T}{T_K} = (1 - f) \quad fK$
- $E = {}^{S_p}/_{p} = \frac{1-f}{p} + f$

→ Still limited by the serial fraction, but it becomes less important as the # of processor grows

Scalability

$E = {}^{S_p}/_{p}$

◎ Scalability is of a parallel system is a measure of its capacity to increase speedup in proportion to the number of processing elements

◎ A parallel algorithm is **scalable** if E_p can be kept constant as P grows

- What would effect E_p?
- $E = \frac{S_p}{p} = \frac{T_s}{pT_p}$
- $T_o = pT_p - T_s$
- $E = \frac{1}{1+\frac{T_o}{T_s}}$

Next Class

Introduction or Parallel Computing 3